Trinity College Dublin
Coláiste na Tríonóide, Baile Átha Cliath
The University of Dublin

Adaptive Applications CS7IS5

Lumino: Holistic User Modeling for Career-Centric Job Recommendations

Team Bosch – Final Report

Ajinkya Taranekar (ID: 24347965)	taraneka@tcd.ie
Lara Hombrecher (ID: 25337624)	hombrecl@tcd.ie
Mohammed Owais Khan (ID: 25332207)	mkhan5@tcd.ie
Shreshtha Kamboj (ID: 23364317)	kambojs@tcd.ie
Xizhen Luo (ID: 25332297)	luox1@tcd.ie

Abstract

As job search and recruitment processes nowadays rapidly shift to online platforms, digital job recommendation systems are getting more and more important. However, candidates' true abilities cannot be captured and reflected due to existing platforms that often rely on opaque and keyword-based matching mechanisms. This leads to inaccurate matches and makes recruitment workflows less efficient.

To address existing challenges, we present Lumino, a scrutable and adaptive job recommendation application that places wholistic user modelling at its core. Lumino constructs a dynamic digital-twin of each candidate using structured knowledge graphs derived from resumes, interactive questioning, and user feedback. Unlike traditional systems, it models both skills and domain knowledge within a hierarchical taxonomy, enabling precise and context-aware matching between candidates and job requirements.

A key contribution of this work lies in the integration of adaptability and scrutability. The system continuously evolves with user interactions, updating preference signals, skill weights, and behavioural interests to refine recommendations over time. At the same time, every recommendation is fully explainable through explicit graph-based relationships, allowing both candidates and recruiters to understand why a match occurs. Users can inspect and modify their own profiles, ensuring control over their representation and reducing the "black-box" effect common to existing platforms.

Technically, Lumino is implemented in the React + FastAPI + Neo4j + SQLite + LiteLLM framework. It combines large language model (LLM)-based information extraction with deterministic graph matching implemented in Neo4j. This hybrid approach ensures both flexibility and traceability by separating interpretation from scoring. Furthermore, features such as evidence-weighted skill, interactive profile refinement, and candidate exploration on the recruiters' side further improve the system's effectiveness.

Overall, Lumino demonstrates how transparent modelling and adaptive interaction can significantly improve the quality, reliability, and usability of job recommendation systems, thus benefiting both job seekers and recruiters.

Keywords: *User Modeling, Job Recommendations, Career Development, Scrutability, Talent Acquisition, Adaptability.*

1 Introduction

The introduction of the Internet has changed how we conduct our daily lives, including how we find jobs. Over the past few years, the job search and application process has increasingly shifted to digital platforms such as Indeed, LinkedIn, or IrishJobs, which help job seekers across multiple domains find the best matches for their career. In the context of evolving economic conditions and an increasingly competitive job market, access to effective job platforms has become more critical than ever.

Already in a 2015 study, 79% of job seekers declared that they sourced online information [5] while in 2018, applications through these online portals accounted for $\frac{1}{5}$ of the hirings worldwide [6]. This shows that the impact of online job portals is large.

1.1 Project Motivation

Unfortunately, traditional recruitment tools (e.g. LinkedIn) operate largely as black boxes [1]. Matching of candidates to potential jobs is done via keyword matching and non-transparent algorithms. This creates a disconnect between the career potential of candidates and the company's need for ideal employees. Several problems arise from this:

Candidates do not feel in control of their own career development. Many job recommendations feel irrelevant to their actual expertise, and they lack the insight to understand which of their skills are highly valuable and can enable them to contribute meaningfully in a given role. Likewise, the decision process feels arbitrary to the candidates, and pure keyword matching does not uncover true skills hidden in the execution of relevant projects.

For recruiters, the way current job matching systems are set up, it is hard to accurately estimate a candidate's skills or fit for a team. This leads to an over-reliance on many rounds of technical and personal fit interviews, taking up a lot of costly resources in the recruiting process. All of these obstacles can elongate the hiring processes and complicate the selection of the ideal candidate.

To alleviate the problems described above, we propose Lumino, a scrutable, adaptive job recommen-

dation application. Lumino builds a transparent user model and helps refine candidates' profiles with personal and in-depth questions. Therefore, it enables both candidates and recruiters to influence the job matching process and achieve the best result for all parties involved.

1.2 Project Goal

Taking into account all the shortcomings mentioned above, the goal of our project is to design an adaptive, transparent, and user-controllable job recommendation application that improves user experience and stands out from numerous similar products. These advantage services can be classified into the following aspects:

Exact Match: The primary goal is to provide exact skill-job matching rather than simple keyword-matching. Taking a possible scenario, for example, a computer science graduate applies for jobs in "data science." Traditional platforms may recommend generic roles such as "data analyst" or "business analyst" based on keywords in his CV. However, our application identifies that the user has more experience in deep learning and probabilistic modelling, so it recommends jobs such as "Machine Learning Engineer" or "AI Research Assistant", which align better with the user's actual abilities.

Skill Insights and Career Potential: Lumino's other objective is to provide users with meaningful insights into their skills, therefore helping them understand both their strengths and improvement space. For example, a user receives job recommendations but may notice that many roles require "distributed systems experience." Our application highlights that, although the user has strong programming skills, they lack evident experience in distributed networks. Furthermore, our system explains which past experiences contribute to the current profile and points out relevant skill gaps. Hence, users have a better understanding of why certain jobs are recommended or not, turning the system from a programmed tool into a career consultant.

Profile Amend and Refine: Our application allows users to continuously refine their personal model through simple interaction. For example, a user initially receives recommendations for frontend development roles; however, he recently developed a new interest in backend engineering. Our application allows the user to modify his preferences and provide additional input (e.g., recent projects or learning goals). The recommendation system will accordingly update its internal representation and shift recommendations. With this evolutionary refinement, users are not constrained by outdated or incomplete profiles, so the major limitation of static job recommendation platforms can be solved.

Enhanced Transparency: An important goal of this application is to make the recommendation process interpretable and transparent to all users. Instead of simply presenting a list of jobs, our application shows a breakdown of why a particular job is recommended with detailed visible weights of several factors. This level of explainability builds trust and allows users to critically evaluate the reasoning of the system, reducing the "black-box" effect seen in traditional platforms.

See Beyond the Resume: As a bilateral job recommendation system, Lumino also helps recruiters in decision-making. Recruiters review a candidate recommended by Lumino and, instead of purely relying on a sheet, they see a structured profile including inferred competencies, skill confidence levels, and explanatory reasoning. This reduces uncertainty and helps recruiters screen candidates more efficiently, which also potentially reduces the number of interview rounds.

1.3 Project Overview

Lumino is an adaptive job recommendation application designed to break through the limitations of traditional online recruitment platforms. Adaptivity and scrutability in our settings are defined in Section 2.1.

Unlike conventional systems that rely heavily on keyword matching and invisible algorithms, Lumino introduces a transparent and interactive framework for modelling users and offering job recommendations, achieving adaptivity and scrutability. At the very beginning, Lumino collects and structures user information through both direct input (e.g., resume) and adaptive questioning (interactive Q&A). This allows the system to build an abundant and more accurate representation of the user compared to static CV-based approaches.

Then, job recommendations are generated using a matching mechanism that considers multiple factors beyond keywords, such as contextual relevance of skills and demonstrated experience. Importantly, the system provides explanations for each recommendation, making the process interpretable.

As for flexibility, users can refine their profiles and preferences over time, as the system automatically adapts based on user's behaviour and feedback. This creates a virtuous feedback loop that continuously improves the quality of the recommendation.

1.4 Report Outline

The rest of this report is structured as follows:

- Section 2 discusses the adaptability and scrutability of our application.
- Section 3 explains the design of the user model in our application.
- Section 4 talks about how data are managed in this project.
- Section 5 explains the details of the implementation and the technology used in our project.
- Section 6 points out the main challenges we encountered and solutions.
- Section 7 displays our achievements.
- Section 8 then discusses the results and provides an outlook.

2 Adaptability and Scrutability

2.1 Definitions

As Lumino is an adaptive and scrutable application, we define and refined the notions of what adaptability and scrutability mean to our application and in the context of job recommendation systems. The definitions that represent our project are presented below.

2.1.1 Definition of Adaptability

The system must adapt to the evolving skills and changing career goals of the user, moving beyond static resume data to capture real-time professional interests. Taking into account the definition of personalization, this adaptability ensures that each candidate is matched to their individual interests and skills.

2.1.2 Definition of Scrutability

The 'why' of every decision must be understandable. Applicants see how a job fits their career plan, and recruiters see the reasoning behind a candidate's compatibility score beyond just their previous job titles. The notion of scrutability also necessitates an interaction from the user, and candidates are prompted to refine their resumes. In this way, they can inspect their modelling, giving them more control over how they are perceived.

2.2 Implementation

To ensure that our definitions of adaptability and scrutability are reflected in Lumino, we have designed Lumino around these principles, making it a job recommendation system with adaptability and scrutability at heart.

A user-specific ranking, according to the best matches and the job interests of an user, serves as the adaptive component of our system. Through this feature, job recommendations are targeted towards both towards domains of interests and matching. This presents users with their most relevant and also most likely options. The likeliness hereby is computed using matching edges from the knowledge graphs representing both user and job.

For the scrutability, these knowledge graphs also play a crucial role. With the connections between a job seeker and a job description, their match can be explained. A user can explore how and why they are matching to a job through the matching edges between the two graphs. In their user profile, they can also inspect the knowledge graph that was generated about them, including everything that the system knows about them. Giving full control over their own model to the user is a central part of our view

of scrutability. Therefore, users can also edit the knowledge graph and extend it, give more detail on projects they have done or frame their achievements in a different light to change how they are perceived by the system.

A technical description of these mechanisms will be presented in the following chapters.

3 User Modelling - Taxonomy Design

3.1 Overview

A core architectural challenge in Lumino is ensuring that skills and domains extracted from heterogeneous sources (a candidate's resume or a recruiter's job posting) can be compared with regards to reliability and transparency. String labels extracted from natural language are inherently inconsistent. One resume may list `'Postgres,'` another `'PostgreSQL,'` and a job posting may require `'PostgreSQL 15'`. The absence of controlled vocabulary would make deterministic matching difficult, which could threaten Lumino's core guarantee of scrutability.

To deal with this, Lumino introduces a structured taxonomy for both skills and domains. The taxonomy serves three purposes, establish a canonical naming convention across all ingested entities, organise them into a hierarchy that supports multi-granularity reasoning, and form the structural backbone over which it `MATCHES` edges. Lumino's main components of explainable matching are created during ingestion (such as resumes or job postings).

3.2 Skill Taxonomy

3.2.1 Hierarchical Structure

Skills in Lumino are structured in a hierarchical way where each skill is situated within a three-level hierarchy:

```
SkillCategory
   SkillFamily  (e.g. "Web Frameworks")
         Skill  (e.g. "FastAPI", years=2, level="intermediate")
```

The `SkillCategory`, which is the root node in this hierarchy, represents the broadest grouping, such as *Backend Development*, *Machine Learning*, or *DevOps*. The node below it, the `SkillFamily`, groups skills that are directly substitutable or closely related in practice. For example, *Web Frameworks* (containing FastAPI, Django, Flask) or *Graph Databases* (containing Neo4j, Amazon Neptune). The leaf node, `Skill`, contains the quantitative properties for an individual, such as years of experience and a self-assessed or inferred proficiency level.

There were two reasons for choosing a hierarchical system over a flat label approach. First, Cypher path queries (search/query graphical data) are inherently hierarchical, descending from a `User` node through `SkillCategory`, and `SkillFamily` to a terminal `Skill` node. No joins are required and the traversal path itself constitutes a human-readable audit trail. Second, the family level enables partial-match reasoning, which enables future extensions to award partial credit where a candidate possesses a skill belonging to the same family as a job requirement, even in the absence of an exact canonical match (see Section 3.6).

3.2.2 Role of the Taxonomy in Ingestion

During LLM extraction, the system prompt embeds the full `SKILL_TAXONOMY`: a predefined mapping of skill names to their canonical family and category labels. The LLM is instructed to normalise every skill it encounters to a canonical entry from this taxonomy before writing it to the extraction schema. If no canonical entry exists for an encountered skill, the LLM creates a new entry under the most appropriate existing family.

Notably, the taxonomy classifies *what a skill is*, not *how important it is*. Importance is a property of the `JobSkillRequirement` node, not of the taxonomy node itself. This separation ensures that the taxonomy remains stable across different job postings, while requirement-level annotations applied during matching can vary per role.

3.3 Domain Taxonomy

Domains represent areas of industry or problem-space expertise. It constitutes the contextual knowledge a candidate brings from having worked in a particular sector, distinct from the technical tools they have used. The domain taxonomy is identical to the skill hierarchy:

```
DomainCategory
   DomainFamily  (e.g. "FinTech")
         Domain  (e.g. "Payment Systems", years=3, depth="deep")
```

`DomainCategory` captures high-level sectors such as *Financial Services*, *Healthcare*, or *Enterprise Software*. `DomainFamily` groups related fields within a sector. For instance, *FinTech* (containing Payment Systems, Lending Infrastructure, Regulatory Compliance) would come under *Financial Services*. The leaf `Domain` node carries years of experience and a depth indicator (`'surface'`, `'working'`, `'deep'`) that reflects the candidate's level of contextual immersion.

The inclusion of a separate domain hierarchy, rather than treating domain knowledge as just another skill, is a deliberate modelling decision. Domain expertise is harder to transfer, less visible on a CV, and frequently undermined by keyword-based systems. By keeping it as a separate class, Lumino treats domain fit as an explicit, traceable signal rather than an implicit one buried in keyword overlap.

3.4 Canonical Naming and Normalisation

The `MATCHES` edge between a user's `Skill` node and a job's `JobSkillRequirement` node is created during ingestion using the following matching criterion:

```
toLower(trim(s.name)) = toLower(trim(req.name))
```

This is an exact canonical name comparison after case-folding and whitespace trimming. The decision to use exact matching rather than fuzzy string similarity was deliberate. Fuzzy matching (e.g. Levenshtein distance or embedding cosine similarity) would allow `'Postgres'` to match `'PostgreSQL,'` but could also silently merge skills that are genuinely distinct. For example, `'PyTorch'` and `'TensorFlow'` could score a partial match despite being non-substitutable in a production codebase.

By enforcing exact canonical matching, Lumino leaves the normalisation responsibility to the LLM extraction step, where it belongs. The LLM is instructed to resolve ambiguous or abbreviated names (such as `'Postgres'`, `'PG'`, `'psycopg2'`) to the single canonical entry `'PostgreSQL'` defined in the taxonomy before the node is written to the graph. If normalisation is applied consistently during ingestion, the matching criterion at query time can remain exact, preserving the full traceability of every `MATCHES` edge.

A practical consequence of this design is that canonical name mismatches produce zero-credit misses rather than partial matches. This is considered a feature rather than a limitation. A miss surfaces as a visible gap in the match explanation, prompting either the candidate to update their profile vocabulary or the recruiter to review their job requirement labels, both of which improve the quality of the graph over time.

3.5 Taxonomy as the Foundation of Scrutability

The taxonomy is not merely an organisational convenience but it is the structural prerequisite for Lumino's scrutability guarantee, as shown in the example in Figure 1.

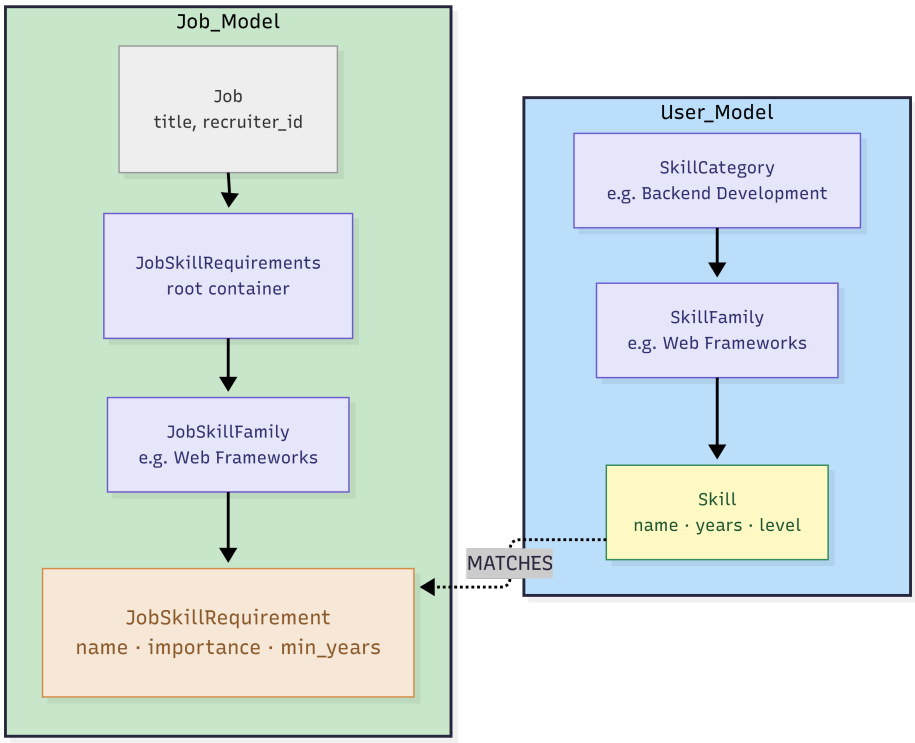

Figure 1: An example detailing the matching process

Every match reason presented to a user is a concrete, traversable path in the knowledge graph:

```
User → HAS_SKILL → FastAPI → MATCHES → FastAPI (JobSkillRequirement) → [Job]
```

This path exists because both the user's `Skill` node and the job's `JobSkillRequirement` node were normalised to the same canonical label, `'FastAPI,'` by the taxonomy during ingestion. Without the taxonomy enforcing a shared vocabulary, these two nodes would carry whatever surface form the LLM happened to extract, `'fastapi'`, `'Fast API'`, `'FastAPI framework'`, and no deterministic edge could bridge them.

The match percentage a user sees is therefore not an opaque confidence score but rather the ratio of taxonomy-grounded requirement nodes covered by taxonomy-grounded profile nodes. The taxonomy is what makes this ratio auditable.

3.6 Extensibility and Future Considerations

The current matching criterion operates at the leaf level. A `Skill` node must carry the same canonical name as a `JobSkillRequirement` node to generate a `MATCHES` edge. However, the three-level hierarchy was designed with a future extension in mind, *family-level partial matching*.

Under this extension, if a candidate has no skill in a required family at all, they receive zero credit. If they have at least one skill in the same `SkillFamily` as a required skill (for example, Flask in the *Web Frameworks* family when Django is required), the system could award a configurable partial credit. This would allow Lumino to distinguish a complete family gap from a within-family substitution without abandoning exact matching at the canonical level. The taxonomy hierarchy already supports this query pattern natively and enabling it would require only a change to the matching Cypher queries and an additional display label in the match explanation.

A secondary extension concerns taxonomy maintenance. As new skills emerge or domain boundaries shift, the taxonomy will require updates. The current approach, a static mapping embedded in the LLM system prompt, is appropriate for a prototype but does not scale. A production system would expose a taxonomy management interface allowing administrators to add canonical entries, define aliases, and reclassify families without modifying the system prompt.

4 Data Management

Lumino has two primary user entities: candidates and recruiters. Each entity inputs structured data (resume/CV for a candidate and job posting for a recruiter) that is transformed into a semantic knowledge graph. The knowledge graph can be interpreted as a digital twin for the candidate and a visual representation of the job for the recruiter. This forms the backbone of the transparent matching algorithm.

4.1 Raw Dataset

To test the matching algorithm and the performance of the application, sample jobs and resumes are uploaded. For the candidate user entity, the resumes of the group members were selected, and the NLP based extraction mechanism extracts skills, experience, education, projects, certifications, etc. to build a digital twin for the candidate.

For the recruiter user entity, the sample job postings were extracted from a large dataset on Kaggle [7]. From the 2024 Linkedin Jobs dataset, a subset of semantically significant attributes was extracted, discarding redundant or incomplete fields that would not contribute information value to the knowledge graph representation of job postings.

4.2 Processed Dataset

The dataset has a diverse set of job roles, structural richness, and comprises three different subsets; one for job skills, one for job summary, and one for the Linkedin Job posting. Rather than retaining all content, semantically relevant columns - *job_link, job_title, company, job_location, job_level, and job_type* were selected.

The three dataframes are joined on a common key *job_link* producing a unified $'merged_df'$ dataframe with all the filtered attributes. A left join ensures that all job postings are preserved even if a corresponding skills or summary entry is absent, avoiding unintentional data loss.

The dataframe is further filtered in two sequential stages: *job_title = Software* and *job_location = New York*, scoping the dataset [8] to a domain and a single well-defined job market. The jobs are manually uplifted on Lumino through recruiter user accounts.

5 Architecture and Technology Stack

5.1 System Overview

At core, Lumino is a graph-based job matching platform. Its architecture is shaped by three properties that most matching systems currently do not posses, by definition they are described as **scrutability** (decisions can be inspected and corrected by the people they affect), **meta-cognition** (the system tracks and acts on its own confidence in what it knows), and **adaptability** (the candidate model and scoring behaviour evolve with use).

Lumino create models for both resume and job posting as a multi-layer knowledge graphs and scores them against each other through deterministic Cypher traversal [2]. The LLM handles only the extraction from natural language type Job posting and User Resume, the graph handles judgement. Every score number maps to a named edge that any actor can inspect.

5.2 Technology Stack

Layer	Technology	Purpose
Web framework	FastAPI 0.116 + Uvicorn	Async REST API
Graph DB	Neo4j 5.28 (Community) [2]	Candidate & job knowledge graphs
Supplementary	SQLite (aiosqlite)	Flags, sessions, checkpoints, analytics
LLM adapter	LiteLLM 1.82	Gemini / Groq / OpenAI / Ollama
Graph viz	pyvis 0.3 + NetworkX 3.4	Self-contained interactive HTML graphs
Frontend	React 18 / TypeScript / Vite	SPA, TailwindCSS, Recharts

Table 1: Core technology stack.

5.3 Core Design Pillars

5.3.1 Scrutability

Scrutability in Lumino means a user can always ask "why" and get a traceable answer, and can always correct the system when that answer is wrong. So, for every matched skill, the system exposes the explicit graph path that produced the given score: from the candidate node through the matched skill to the corresponding job requirement. "Python contributed 0.12" is not a post-hoc approximation; it's based on the calculation done by the system backed by other nodes.

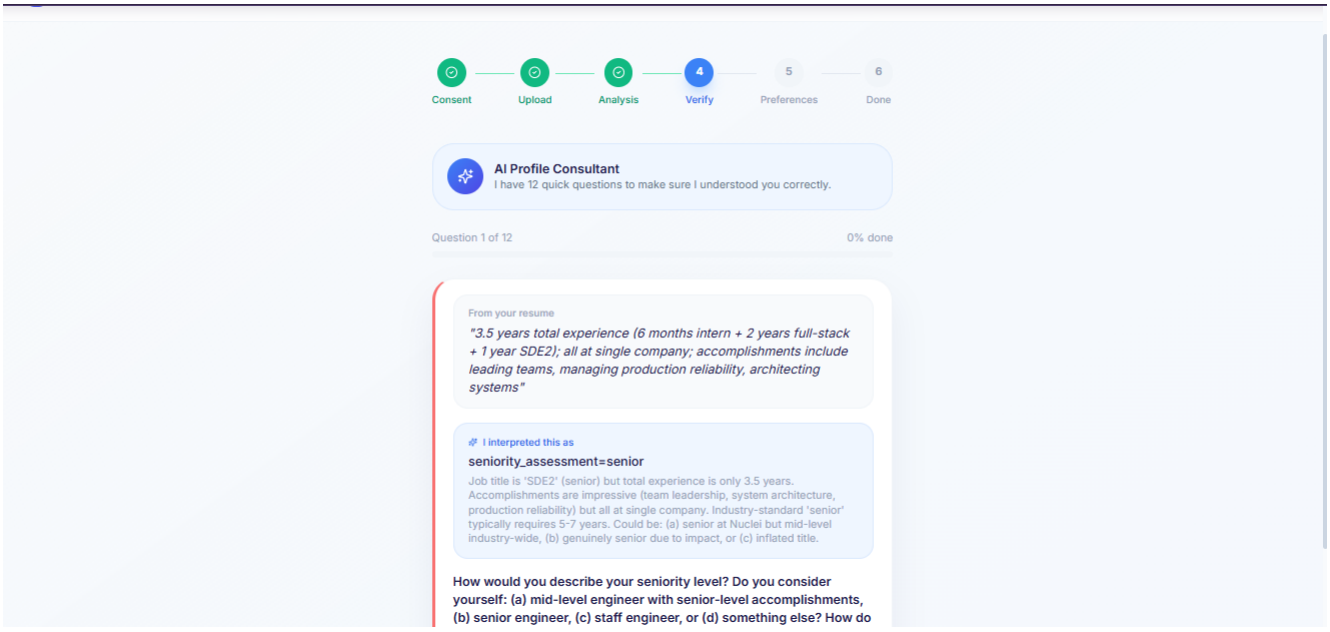

Figure 2: LLM asks for the clarification during extraction.

When a user requests for an explanation, the engine gathers context from the matched nodes, assembles the path traces and evidence tiers, and generates a natural-language breakdown tailored to either the job seeker or the recruiter perspective. For extraction decisions, every LLM inference that lacks direct textual support is stored as an interpretation flag carrying the raw text, the inferred value, an ambiguity reason, and an estimate of how much that ambiguity affects the final score. Candidates resolve these flags through the clarification interview before the graph is marked as verified. The system cannot be silently wrong; it can only be visibly uncertain.

5.3.2 Meta-cognition

The system tracks its own confidence in three concrete ways, and lets those confidence levels influence every score.

Evidence-strength tiers. Every skill node carries an `evidence_strength` property with four levels and corresponding score weights:

Evidence tier	Score weight
multiple_productions	1.00
project_backed	0.80
mentioned_once	0.50
claimed_only	0.30

A skill on a resume with no supporting context earns 0.30. The same skill backed by a project with 5W+H annotations earns 0.80. The gap is intentional: the system encodes how much it trusts its own data.

Profile quality multiplier. The LLM's critical assessment of a profile yields an overall signal rating (*strong / moderate / weak / misleading*), which the system maps to a quality multiplier ranging from 0.5

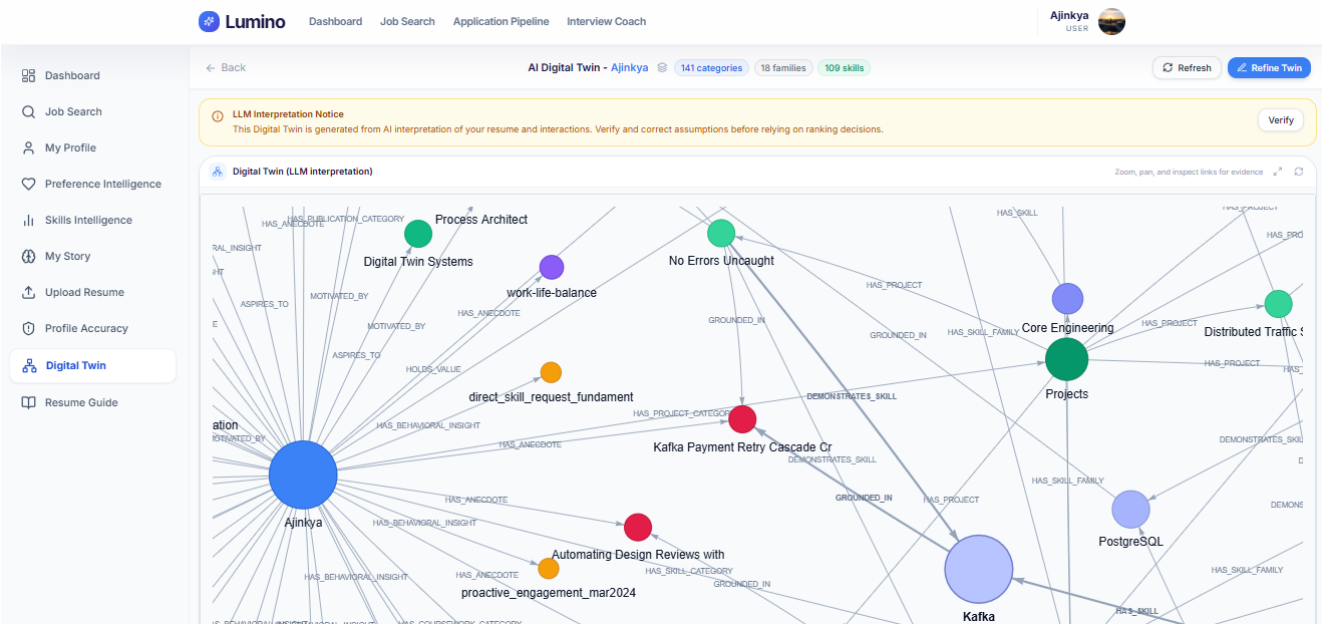

Figure 3: Kafka being backed by a project and a supporting anecdote

to 1.0, with an additional -0.05 applied per identified red flag. A profile assessed as weak cannot score as highly as one assessed as strong, regardless of which skills it claims.

Extraction flags. Each flag stores a confidence level (*high / medium / low*) alongside the ambiguity reason. The graph is marked as verified only once every critical flag has been resolved by the candidate, not by the model.

5.3.3 Adaptability

The candidate model is not static after ingestion. Three feedback channels update it continuously.

Behavioural interest scoring. As a candidate interacts with the platform, their actions, views, likes, bookmarks, applications, and dislikes, are aggregated into per-tag interest scores stored as interest edges in the graph. Each interaction event decays with a half-life of roughly 14 days: $s_{\text{interest}} = \text{clamp}(\sum_e w_e\, e^{-0.05\,\Delta t_e},\ -1,\ 1)$. The score updates live as the candidate interacts with the platform.

Weight recomputation. After every accepted graph mutation, expertise weights are recalculated for all affected skill and domain nodes: $w = \text{clamp}(0.4\,y + 0.3\,p + 0.3\,\ell,\ 0,\ 1)$ where y is years of experience, p is project count, and ℓ is level value. Skills the candidate deepens through the editing session immediately score higher in subsequent matches.

Hybrid score blending. The final score blends the structural graph score with the behavioural interest score:

$$\text{score}_{\text{final}} = \alpha \cdot s_{\text{graph}} + \beta \cdot s_{\text{interest}}, \quad \alpha = 0.7,\ \beta = 0.3 \tag{1}$$

Interest is capped at 30 % so the graph score, which reflects actual qualifications, always dominates. Candidates can override or delete any interest tag from the Interests page to correct drift.

5.4 Candidate and Recruiter Features

5.4.1 Candidate experience

Onboarding begins with a consent disclosure covering what data will be collected, the scope of LLM transmission, and the candidate's GDPR rights. The upload only proceeds after explicit acceptance. After ingestion, the clarification interview resolves interpretation flags and an LLM-driven editing session, framed as a sincere interviewer, populates the Digital Twin layer: six node types (`Motivation`, `Value`, `Goal`, `Anecdote`, `CultureIdentity`, `BehavioralInsight`) that capture what no resume would state.

Before applying to a job, candidates can run a full five-phase practice interview grounded in the job's actual skill and domain requirements, ending with a scored practice scorecard covering technical

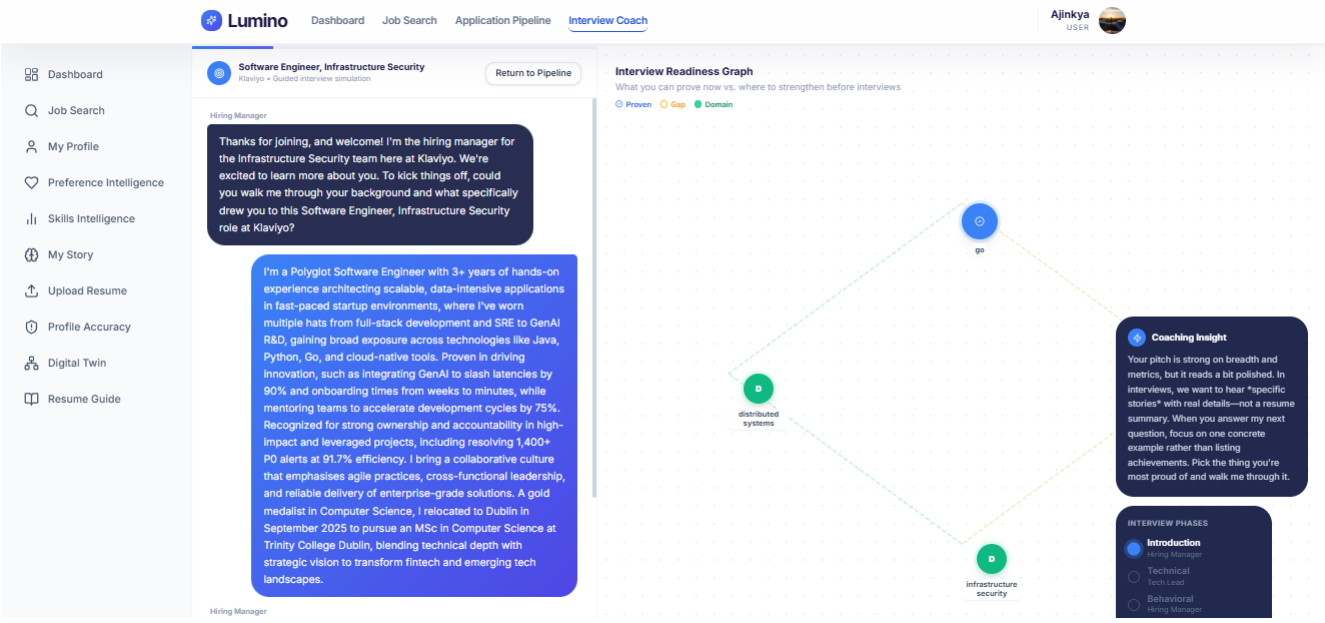

Figure 4: Candidate can practice with Job Profile Model Perspective

depth, behavioural evidence, communication, culture fit, and an overall *strong_yes / yes / maybe / no* recommendation [5].

5.4.2 Recruiter experience

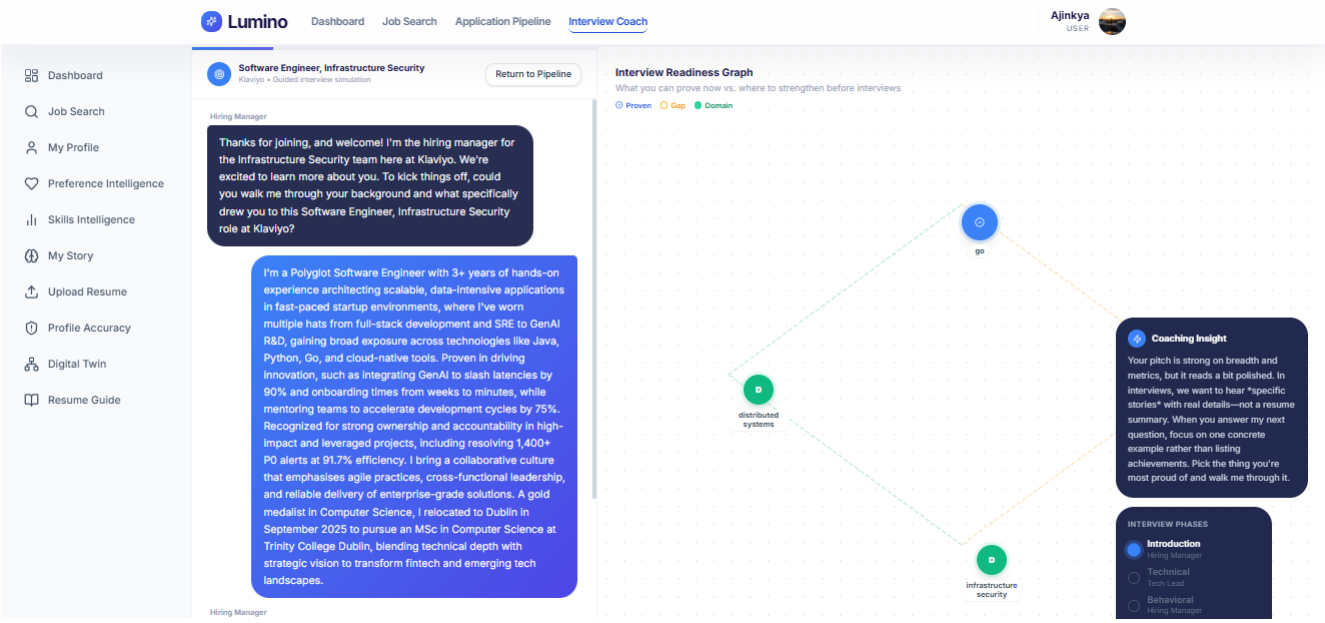

Figure 5: Recruiter can interview with the candidate's digital twin to understand them better

Recruiters see two candidate pools: the applied pool, ranked by match score with a full component breakdown, and the global pool for proactive sourcing of candidates who have not yet applied [6]. Both rank identically using the same engine. The Mirror Interview feature lets a recruiter interrogate any candidate's Digital Twin through a chat interface. Each turn returns a grounded answer, a confidence score, the evidence source, a suggested follow-up, and nightmare questions generated from the candidate's weakest graph signals to surface gaps the match score alone would not reveal.

5.5 Privacy and Data Rights

Candidates can export their complete graph data as a structured JSON file at any time, exercising their right to data portability under GDPR Article 20 [4]. Account deletion triggers a cascade removal of every owned node and relationship without requiring a support request, fulfilling the right to erasure under GDPR Article 17. Both operations are self-service from the candidate profile page.

6 Project Challenges

Based on the three design pillars described in the architecture: scrutability, meta-cognition, and adaptability each created its own engineering problems and challenges, which do not appear in conventional recommender systems. In this chapter we will cover those problems, the decisions made for them, and what challenges still remains open.

6.1 Scalability

Scrutability and adaptive hyperlinking are extensive process, hyperpersonalising for user often leads challenges with engineering problems like scaling for millions of user. Scrutability requires full path tracing for every matched skill, meanwhile adaptive hyperlinking requires `MATCHES` edges to be accurate and up to date for every node in the graph [2]. If we do a naive ranking N candidates against M jobs, it means $O(N \times M)$ full graph traversals.

Three design choices tried to contain the cost. We compute embeddings *once* at ingestion and stored it as pre-built vector indexes. Batch ranking starts from a pre-filtered set, only candidates sharing at least one match edge with the target job, cutting the active pool by orders of magnitude before any traversal begins. Moreover, the frontend caches scored pairs so revisiting a match costs nothing.

6.2 Logic Complexity

6.2.1 Cold-start problem

In recommender systems, cold start problem is a major challenge, here a new candidate arrives with no interaction history, with a possibly sparse resume, and no digital twin nodes. Their score is nearly all technical graph ($0.7 \times s_{\text{graph}}$) with a flat interest baseline. So, rather than waiting for interaction data to accumulate, the system interviews the candidate immediately through the clarification and graph editing sessions to fill what the resume left blank. This helps in keeping LLM Hallucinations in check also building the candidate profile. Concrete questions upgrade evidence strength: "Walk me through a project where you used Python under pressure" moves *claimed_only* to *project_backed*; "What drew you to backend work?" populates motivation nodes. On the job side, the LLM tends to over-classify requirements as *must_have*.

6.2.2 Avoiding over-adaptation

Adaptability can creates its own problem, for example, a candidate who only interacts with Python backend roles gets ranked higher for more of the same, and everything else gets buried for them. So, the interest score is capped at 30 % of the hybrid blend so the structural graph score, which reflects the candidates actual qualifications, always contributes at least 70 %. Candidates can also override or delete any interest tag from the Interests page to correct this drift manually, providing them with full authorithy to adapt to Lumino.

On the other hand, Meta-cognition creates a parallel risk. If say the LLM over-confidently assigns *strong* or *multiple_productions* evidence to skills it only inferred, the evidence-strength tier system loses its meaning. The interpretation flag workflow is the guard, so every uncertain inference stays unverified and carries a low-confidence tag until the candidate confirms it. The architecture supports per-domain weight profiles, but calibrating them requires data the system does not yet collect, example from recuriter side a feedback like did the hire work out? which can be added in future prospects.

6.3 Testing

Testing is tough where LLMs and vectors are involved, moreover, with a graph database, a vector index, and an LLM API all in play, standard unit testing only covers the easy parts. In Lumnio what makes the harder parts testable is the same decision that made scoring explainable, the LLM never touches scoring.

Only the matching engine, weight computation, and score decomposition can be tested by injecting pre-built graphs with no LLM calls at all [7]. LLM extraction tests check schema conformance only, not specific extracted values.

6.4 Versioning and Rollback

Candidates can mutate their graphs extensively through the editing session. A mistaken deletion or an over-eager LLM proposal could silently break a profile with no recovery path. So, we built versioning and rollback system, where Lumino's state spans two databases: Neo4j holds the live graph; SQLite holds flags, session messages, and analytics [2]. A logical rollback must restore both consistently. Based on this each checkpoint serialises the full user or job subgraph as a JSON blob in SQLite. Rolling back to a prior checkpoint deletes the current subgraph, restores all nodes and relationships from the blob, and recalculates expertise weights. Checkpoints are created per mutation batch so the timeline stays readable: one snapshot per "thought", not per low-level graph operation.

6.5 Transparency and Scrutability Limits

Scrutability was the hardest constraint to maintain, because it conflicts with every shortcut. Building the explanation endpoint to generate a narrative from matched node context and path traces is straightforward; making that explanation an accurate trace of the actual scoring computation, not an LLM approximation of it, required keeping the LLM completely out of the scoring path. Moreover, when user edit their profile, LLM edit proposals show the proposed graph mutation but not the reasoning chain behind it which is still in black box.

6.6 Ethical Considerations

Lumnio has taken various Ethical Consideration into motion, since a system that influences who gets interviewed has obligations beyond producing a correct score [1]. LLM extraction bias, associating certain skills with certain demographics, or extracting less from non-standard resume formats, can propagate into the graph and distort match scores [1]. Currently, the frontier models were used for MVP, specialised local LLMs finetuned for extraction based on different resume can be added in future prospects. Currently, the clarification interview gives every candidate the chance to upgrade evidence strength regardless of how their resume was phrased, and score decomposition is fully auditable so recruiters can inspect which edges drove a result. Whether the LLM extracts less from non-standard formats remains an open question that needs evaluation across a broader range of resume styles. Because the scoring formula is public, candidates could game it: claim every skill and rely on the nonzero *claimed_only* weight. The `must_demonstrate` flag on critical job requirements forces the engine to look for project-backed evidence, and a low practice-interview scorecard gives recruiters a reality check against an inflated match score.

7 Results

We present Lumino, a job-matching application that places wholistic user-modelling and the resulting characteristics of adaptability and scrutability at its core.

7.1 Application Overview

Our application has an interface for both job seekers and recruiters. Figures 6 and 7 show the candidate's view of Lumino. We can see the two core characteristics here, Figure 6 shows the scrutability part of the application. The candidate can gain insights about the information about them represented in the user model by inspecting their digital twin knowledge graph. Users also have the option to edit the digital twin and supply more detail. Such an additional detail interview is also shown in the picture.

The adaptive characteristic is shown in Figure 7, where the ranking of highest matching jobs is shown, alongside a breakdown of which area has which percentage of matching. Users have different options of interacting with these ranked job postings, from indicating preferences via liking to investigating the job matching and inspecting the exact graph matching.

Figure 8 shows the recruiter perspective of Lumino. As candidates, recruiters also have the chance to inspect job-candidate matches and supply more details e.g. about the work-culture of their company. Their dashboard shows the open job postings that they have, as well as an overview of well-suited candidates.

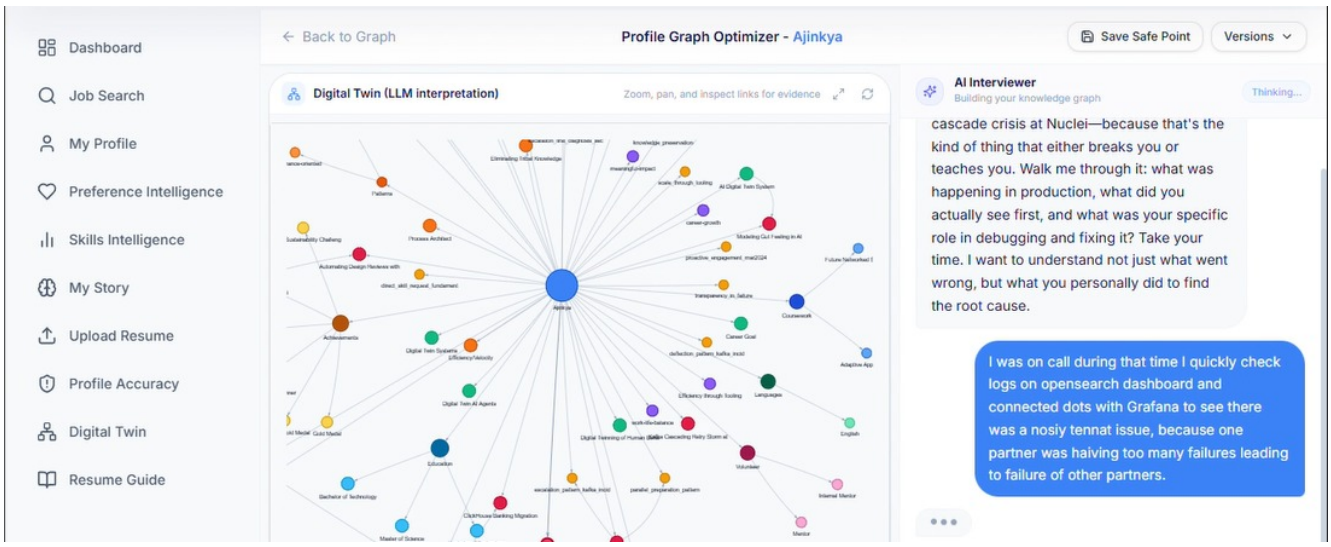

Figure 6: The scrutability view from a candidate's perspective, showing the knowledge graph and an additional clarification interview.

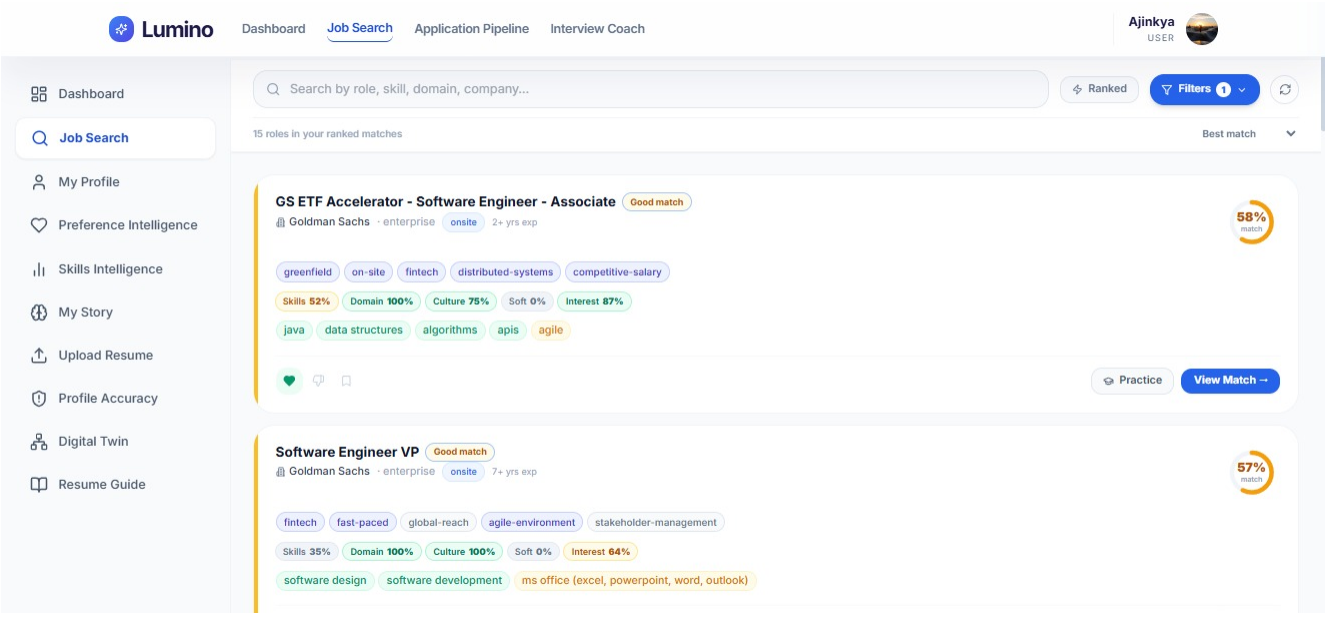

Figure 7: The adaptive view from a candidate's perspective, showing the ranking of best matching job postings.

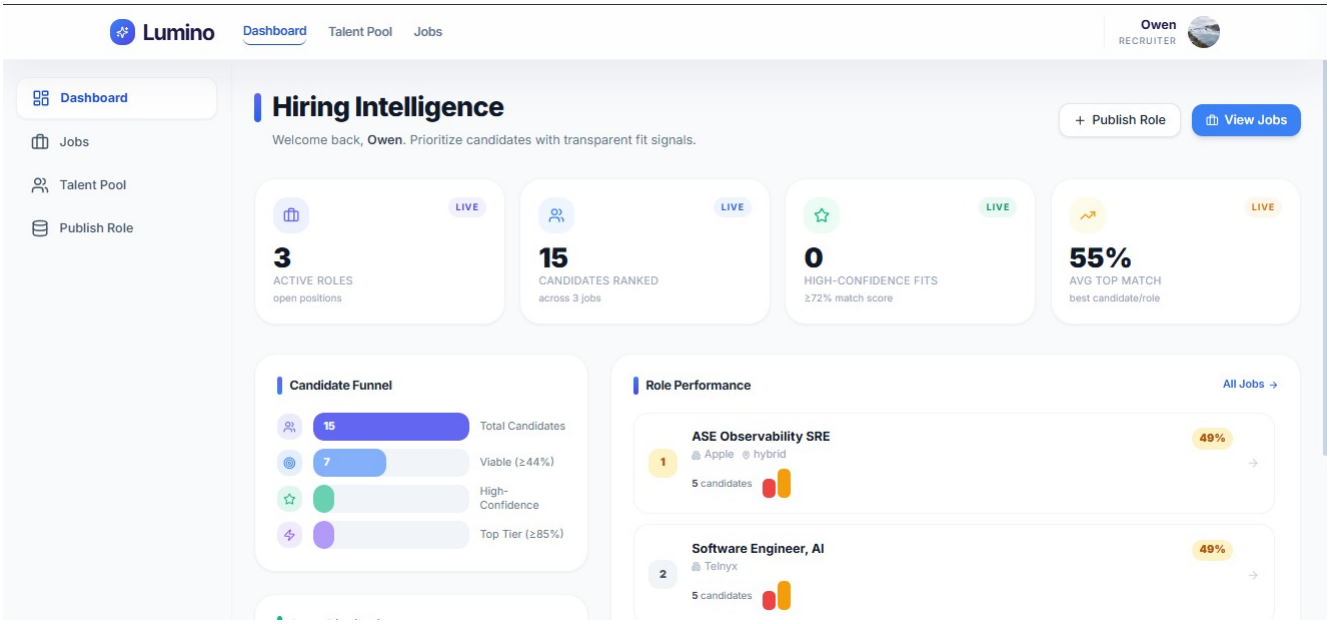

Figure 8: The hiring overview of a recruiter, showing rankings for job matches.

7.1.1 Features

Lumino is a modern, sophisticated, and intuitive application that has a diverse set of features designed to meet a wide range of user needs. From building a dynamic digital twin to preparing you for your next dream job, Lumino helps candidates with everything they need to succeed. It leverages advanced AI models such as *groq, gemini, claude* via secure API integration to intelligently generate and refine a candidate's digital twin. The application has adaptive capabilities that provide personalized recommendations to enhance the profile of the candidate by building an efficient user model. The interview coach feature prepares candidates by conducting a realistic mock interview ahead of their actual interview.

From identifying the ideal candidate for a role to reviewing top-tier talent, Lumino empowers recruiters at every stage of the hiring process. The AI enabled feature modifies a job model knowledge graph to better tailor the requirements for a role.

Lumino models both candidate resumes and job descriptions as structured knowledge graphs and performs semantic matching to produce a relevance score that influences candidate recommendations (ranked based on score), alongside AI-generated summaries that highlight the fundamental reason behind matching and key strengths for each applicant.

7.1.2 Achieved Goals

Lumino achieves the goal of our project, which was to design an adaptive, transparent, and user-controllable job recommendation application. It stands out in a positive way compared to numerous similar products.

Lumino successfully implements skill-level exact matching using canonical taxonomy and `MATCHES` edge traversal, moving beyond keyword matching as explained in Section 3.4.

The skill insight objective is realised through coverage analysis over `MATCHES` edges, where it tries to bridge the gap between a candidate's inferred skill profile and the canonical requirements of a target role.

The iterative profile refinement mechanism was successfully implemented, enabling the users to update their preferences and trigger the re-ranking of recommendations without actually rebuilding the profile from scratch.

Explainability is achieved through Cypher path traversal, where each match is backed by a human-readable reasoning generated by the LLM model based on the graph structure.

Lumino's structured candidate profiles are formed using inferred competency nodes and confidence-weighted skill edges, making it easier for the recruiter to derive a more meaningful summary of a candidate's CV.

8 Discussion

We present Lumino, a job-matching engine with adaptive and scrutable characteristics. We introduce a digital-twin user model and exact graph matching between candidates and jobs for a clear and interpretable skill match. Furthermore, we place an emphasis on the scrutability by giving the user full control over the user model, providing the possibility to add more detail to projects and further specify skills.

As for feasibility demonstration purposes, we have only included jobs and profiles from the Computer Science domain, one big limitation that our application currently has is its lack of diversity. Our taxonomy is currently designed to represent skills in the domain of Computer Science only. So simply adding jobs and resumes from varying backgrounds would not work for our matching system. However, extending the taxonomy to include several professional domains is a difficult process. To correctly represent skills and skill families, a thorough knowledge of the field is necessary, which our team, consisting of computer scientists, is currently lacking, especially considering jobs in the domains of humanities.

Additionally to the limitation of extending the taxonomy, potential limitations arise for users not from a tech background using our application. While our knowledge graphs and interfaces help understand how matches are being made, job seekers not from a tech background might not be able to utilize the information as well. To alleviate this, more explanations or a tutorial could be added to help people not inherently familiar with graph representations get started with our application.

This also ties to the future work to be done on Lumino. To make it applicable for the general public, we want to include more job domains. For this, an extension of the taxonomy as needed, as well as a revised prompt engineering for the ingestion LLM and for the editing of the knowledge graph. As described above, to make the application as accessible as possible, we plan on including help pages and tutorials and plan on evaluating our application in user studies.

References

[1] F. Musrifah and I. Hasanah, "Ethical Implications of AI-Driven Recruitment: A Multi-Perspective Study on Bias and Transparency in Digital Hiring Platforms," *Journal of Management and Informatics*, vol. 4, pp. 599–616, Apr. 2025.

[2] Neo4j, Inc., *Neo4j Graph Data Platform*, version 5, 2024. [Online]. Available: `https://neo4j.com`

[3] European Parliament and Council of the European Union, "Regulation (EU) 2016/679 (General Data Protection Regulation), Article 18: Right to Restriction of Processing," *Official Journal of the European Union*, vol. L 119, pp. 1–88, May 2016. [Online]. Available: `https://eur-lex.europa.eu/eli/reg/2016/679/oj`

[4] European Parliament and Council of the European Union, "Regulation (EU) 2016/679 (General Data Protection Regulation), Article 20: Right to Data Portability," *Official Journal of the European Union*, vol. L 119, pp. 1–88, May 2016. [Online]. Available: `https://eur-lex.europa.eu/eli/reg/2016/679/oj`

[5] T. R. Dillahunt, A. Israni, A. J. Lu, M. Cai, and J. C.-Y. Hsiao, *Examining the Use of Online Platforms for Employment: A Survey of U.S. Job Seekers*, in Proc. CHI Conf. Human Factors in Computing Systems (CHI '21), Yokohama, Japan, May 8–13, 2021. ACM, New York, NY, USA, pp. 1–23. [Online]. Available: `https://doi.org/10.1145/3411764.3445350`

[6] Statista, *Online Recruiting - Statistics & Facts*, [Online]. Available: `https://www.statista.com/topics/2727/online-recruiting/`

[7] Kaggle, *Linkedin Jobs Dataset* [Online]. Available: `https://www.kaggle.com/datasets/asaniczka/1-3m-linkedin-jobs-and-skills-2024/data`

[8] *Data Preprocessing* [Online]. Available: `https://www.kaggle.com/code/shreshthakamboj/adaptivejobpostings`